



Universidad Nacional Experimental de Guayana.

Vicerrectorado Académico.

Coordinación General de Pregrado.

Proyecto de Carrera: Ingeniería en Informática.

Unidad Curricular: Lenguajes y Compiladores.

Semestre 7 – Sección: 1 y 2.

Análisis Sintáctico

Los ASTronautas

“Explorando el Universo de los Lenguajes, un Nodo a la vez”

Profesor:

Ing. Félix Márquez

Estudiantes:

C.I.: V-25.933.680. Albuquerque Sheen

C.I.: V-30.907.427. Antoima Mariangel

C.I.: V-28.475.271. García Carlos

C.I.: V-24.848.424. Varguillas Génesis.

Ciudad Guayana, Julio del 2026.

Introducción

El análisis sintáctico constituye una de las fases fundamentales en el proceso de compilación, siendo responsable de verificar que la estructura del código fuente cumpla con las reglas gramaticales definidas para el lenguaje en cuestión. Esta fase, también conocida como *parsing*, toma como entrada la secuencia de *tokens* generada por el analizador léxico y produce como salida una representación jerárquica del programa, típicamente un Árbol de Sintaxis Abstracta (AST), que será utilizado por las fases posteriores del compilador (Aho et al., 2008). La correcta implementación del análisis sintáctico es esencial porque un programa con errores sintácticos no puede ser compilado ni ejecutado, lo que hace de esta fase un componente crítico en la cadena de herramientas de desarrollo de software.

El presente informe aborda el estudio y desarrollo práctico de los conceptos fundamentales del análisis sintáctico, estructurado en cinco actividades principales, tal como se solicita en el Tema 5 de la asignatura Lenguajes y Compiladores. Las tres primeras actividades se centran en los fundamentos teóricos y su implementación básica. La primera actividad explora el Árbol de Sintaxis Abstracta (AST), su definición, características principales y su implementación mediante ejemplos concretos. La segunda actividad establece una comparación detallada entre las estrategias de análisis LL y LR, incluyendo sus diferencias teóricas, evidencias de uso actual y su implementación práctica para un lenguaje L definido específicamente. La tercera actividad presenta un resumen de los principales generadores de analizadores sintácticos disponibles, junto con las técnicas más utilizadas para la gestión y recuperación de errores sintácticos.

Las actividades 4 y 5 representan un nivel de complejidad superior, integrando los conceptos teóricos en aplicaciones prácticas más elaboradas. La cuarta actividad consiste en un experimento de rendimiento donde se implementan parsers para un subconjunto de Docker Compose (basado en YAML) en tres lenguajes diferentes (C con Flex/Bison, C# con ANTLR, y Python con ANTLR), midiendo y comparando sus tiempos de ejecución. La quinta actividad propone el desarrollo de un asistente de programación híbrido (UnegScript) que combina técnicas tradicionales de compiladores (lexer basado en expresiones regulares y parser recursivo descendente) con un mecanismo de respaldo a inteligencia artificial para la corrección de errores léxicos y sintácticos.

Según Hopcroft, Motwani y Ullman (2007), el análisis sintáctico es la fase donde se verifica que la secuencia de *tokens* cumple con la gramática del lenguaje, construyendo una estructura de árbol que refleja la jerarquía del programa. Esta verificación se realiza mediante algoritmos deterministas que, dependiendo de la estrategia elegida (descendente o ascendente), recorren la entrada de manera predictiva o reductiva, respectivamente. El conocimiento de las gramáticas libres de contexto, los conjuntos FIRST y FOLLOW, y las técnicas de eliminación de ambigüedad y recursión se convierten en herramientas indispensables para el diseñador de compiladores.

ACTIVIDADES:

1, 2 Y 3

Actividad 1: Árbol de Sintaxis Abstracta (AST)

1.1. Definición

Un Árbol de Sintaxis Abstracta (AST) es una representación intermedia del código fuente que abstrae los detalles sintácticos para retener únicamente la estructura esencial del programa. A diferencia del árbol de parsing concreto, el AST elimina elementos como paréntesis, punto y coma, y palabras clave que no afectan la semántica del programa (Aho et al., 2008).

El AST es la estructura de datos central que utilizarán las fases posteriores del compilador: análisis semántico, optimización y generación de código. Su diseño facilita el recorrido para el análisis de tipos, la verificación semántica y la generación de código, permitiendo algoritmos eficientes de visitante (visitor pattern).

1.2. Características

Característica	Descripción
Abstracción Sintáctica	Omite elementos gramaticales superfluos (paréntesis, separadores) y conserva solo la estructura operacional y lógica del programa
Jerarquía Semántica	La estructura del árbol refleja la jerarquía de operaciones y constructos del lenguaje. Los nodos internos representan operaciones o estructuras de control, mientras que las hojas representan operandos, literales o identificadores
Facilidad de Procesamiento	Su diseño simplifica el recorrido para el análisis de tipos, la verificación semántica y la generación de código
Independencia de la Sintaxis	Dos programas con diferente estilo de codificación pero idéntica semántica producirán AST equivalentes

1.3. Ejemplos de AST

Ejemplo 1: Expresión Aritmética

Código Fuente	AST Resultante
(5 + 3) * 2	<pre> ===== EJEMPLO 1: Expresión Aritmética Código fuente: (5 + 3) * 2 ===== ÁRBOL DE SINTAXIS ABSTRACTA - EJEMPLO 1 ===== ├ operador_binario: * (línea 1) ├ operador_binario: + (línea 1) ├ ┬ numero: 5 (línea 1) ├ ┬ numero: 3 (línea 1) ├ ┬ numero: 2 (línea 1) </pre>

Representación textual del AST:

```

[*]
/ \
[+] [2]
/ \
[5] [3]

```

Explicación: La expresión `(5 + 3) \* 2` se representa como un árbol donde la raíz es el operador `\*`, el hijo izquierdo es la operación `+` con sus operandos `5` y `3`, y el hijo derecho es el literal `2`. Esta estructura refleja la precedencia de operadores.

Ejemplo 2: Sentencia Condicional

Código Fuente	AST Resultante
if (x > 0) { y = 1; }	<pre> ===== EJEMPLO 2: Sentencia Condicional IF Código fuente: if (x > 0) { y = 1; } ===== ÁRBOL DE SINTAXIS ABSTRACTA - EJEMPLO 2 ===== ├ if (línea 1) ├ ┬ operador_comparacion: > (línea 1) ├ ┬ variable: x (línea 1) ├ ┬ numero: 0 (línea 1) ├ ┬ asignacion: = (línea 1) ├ ┬ variable: y (línea 1) ├ ┬ numero: 1 (línea 1) </pre>

Representación textual del AST:

```
[If]
 /  \
[>] [Assign]
 /  \ /  \
[x] [0] [y] [1]
```

Explicación: La sentencia `if` se representa como un nodo con dos hijos: la condición (`x > 0`) y el cuerpo de la sentencia (`y = 1`). La condición es una operación de comparación y el cuerpo es una asignación.

1.4. Implementación

La implementación del AST se realizó en Python, utilizando una estructura de nodos que permite representar diferentes tipos de constructos del lenguaje.

Código de la implementación:

```
class NodoAST:
    """
    Clase que representa un nodo del Árbol de Sintaxis Abstracta.
    """
    def __init__(self, tipo, valor=None, hijos=None, linea=0, columna=0):
        self.tipo = tipo      # 'operador', 'numero', 'variable', 'asignacion', 'if', etc.
        self.valor = valor    # '+', 5, 'x', etc.
        self.hijos = hijos if hijos is not None else []
        self.linea = linea    # Para reporte de errores
        self.columna = columna # Para reporte de errores
    def __str__(self, nivel=0):
```

```

indent = " " * nivel

resultado = f"{indent} └─ {self.tipo}"

if self.valor is not None:

    resultado += f": {self.valor}"

if self.linea > 0:

    resultado += f" (línea {self.linea})"

    resultado += "\n"

for hijo in self.hijos:

    resultado += hijo.__str__(nivel + 1)

return resultado

```

Resultados de la ejecución:

Ejemplo	Código Fuente	Estado	Observación
1	(5 + 3) * 2	Correcto	AST generado con operador * como raíz
2	if (x > 0) { y = 1; }	Correcto	AST generado con nodo if y sus hijos
3	while (i < 10) { i = i + 1; }	Correcto	AST generado con nodo while y sus hijos

Actividad 2: Comparación entre Análisis LL y LR

2.1. Definición y Características Generales

El análisis sintáctico se puede realizar mediante dos estrategias fundamentales: descendente (LL) y ascendente (LR). Ambas estrategias determinan si una cadena de tokens pertenece al lenguaje definido por una gramática libre de contexto, pero lo hacen desde perspectivas opuestas (Hopcroft, Motwani & Ullman, 2007).

Análisis LL (Left-to-right, Leftmost derivation): Es un análisis descendente que comienza desde el símbolo inicial y aplica derivaciones por la izquierda. Es más sencillo de implementar manualmente mediante recursión descendente.

Análisis LR (Left-to-right, Rightmost derivation): Es un análisis ascendente que comienza desde los tokens y aplica reducciones construyendo la derivación por la derecha en reversa. Maneja gramáticas más complejas.

2.2. Tabla Comparativa LL vs LR

Criterio	Análisis LL (Descendente)	Análisis LR (Ascendente)
Estrategia de Construcción	Top-Down: Parte del símbolo inicial y expande los no terminales	Bottom-Up: Parte de los tokens y los reduce a no terminales
Tipo de Derivación	Por la Izquierda (Leftmost)	Por la Derecha (Rightmost) en reversa
Potencia Expresiva	Menos Potente: Limitaciones con recursión izquierda	Más Potente: Maneja gramáticas más complejas
Complejidad de Implementación	Más Sencilla: Recursión descendente manual	Más Compleja: Tablas de análisis automáticas
Mecanismo de Decisión	Predictivo: Basado en conjuntos FIRST y FOLLOW	Reducción basada en contexto y pila de análisis
Manejo de Errores	Más Sencillo de Localizar	Más Complejo de Localizar

2.3. Evidencia de Uso Actual

Estrategia	Herramientas Representativas	Aplicaciones Típicas
LL	ANTLR, JavaCC, parsers recursivos descendentes	Lenguajes de dominio específico (DSL), herramientas de análisis en IDEs
LR	Bison, Yacc, SLY, Menhir	Compiladores de lenguajes de propósito general (C, C++, Java, Python)

2.4. Implementación para el Lenguaje L

Se definió un lenguaje L simple que consiste en listas de identificadores separados por comas y terminados en punto y coma. Ejemplo válido: a, b, c;.

Gramática LL(1):

lista_ids -> id cola_lista_ids

cola_lista_ids -> , id cola_lista_ids

cola_lista_ids -> ;

Gramática LR:

Para el análisis ascendente (LR), se definió la siguiente gramática para el lenguaje L:

lista_ids -> ids PUNTO_Y_COMA

ids -> ids COMA ID

ids -> ID

Donde:

- lista_ids es el símbolo inicial
- ids representa una lista de identificadores
- PUNTO_Y_COMA es el terminal ;
- COMA es el terminal ,
- ID es un identificador

2.5. Resultados de la Implementación

2.5.1. Resultados del Parser LL

Prueba	Entrada	Resultado Esperado	Resultado Obtenido	Estado
1	id, id;	VÁLIDO	VÁLIDO	☐
2	id, id, id;	VÁLIDO	VÁLIDO	☐
3	id;	VÁLIDO	VÁLIDO	☐
4	id, id, id, id;	VÁLIDO	VÁLIDO	☐
5	id, ;	INVÁLIDO	INVÁLIDO	☐
6	id, id	INVÁLIDO	INVÁLIDO	☐
7	, id;	INVÁLIDO	INVÁLIDO	☐
8	id id;	INVÁLIDO	INVÁLIDO	☐
9	id, id, ;	INVÁLIDO	INVÁLIDO	☐

Tasa de éxito: 100% (9/9 pruebas)

2.5.2. Resultados del Parser LR

Prueba	Entrada	Resultado Esperado	Resultado Obtenido	Estado
1	a, b, c;	VÁLIDO	VÁLIDO	☐

2	x, y, z, w;	VÁLIDO	VÁLIDO	☐
3	id1;	VÁLIDO	VÁLIDO	☐
4	a, b;	VÁLIDO	VÁLIDO	☐
5	a, b, c, d, e;	VÁLIDO	VÁLIDO	☐
6	a, , b;	INVÁLIDO	INVÁLIDO	☐
7	a, b, ;	INVÁLIDO	INVÁLIDO	☐
8	a b c;	INVÁLIDO	INVÁLIDO	☐
9	;	INVÁLIDO	INVÁLIDO	☐
10	a, b, c	INVÁLIDO	INVÁLIDO	☐

Tasa de éxito: 100% (10/10 pruebas)

2.5.3. Comparativa de Resultados

Parser	Pruebas Realizadas	Pruebas Exitosas	Tasa de Éxito
Parser LL (Rekursivo Descendente)	9	9	100%
Parser LR	10	10	100%

Actividad 3: Generadores de Analizadores Sintácticos

3.1. Resumen de Generadores

Los generadores de analizadores sintácticos (también llamados metacompiladores) son herramientas que, a partir de una especificación formal de una gramática (generalmente en formato BNF o EBNF), generan automáticamente el código fuente de un *parser*

completo. Estas herramientas son fundamentales en el desarrollo de compiladores y procesadores de lenguajes (Parr, 2013).

Generador	Tipo de Parser	Lenguajes de Salida	Características Destacadas
ANTLR	LL(*)	Java, Python, C#, Go, C++, JavaScript, Swift	Soporte para múltiples lenguajes destino, genera código legible, tiene entorno de desarrollo integrado (ANTLRWorks)
Bison	LALR(1)	C, C++, Java	Sucesor de Yacc, estándar en entornos Unix/Linux, altamente eficiente
JavaCC	LL(k)	Java	Generador específico para Java, soporta <i>lookahead</i> arbitrario
Flex + Bison	LALR(1)	C	Combinación clásica, herramientas maduras y de alto rendimiento
SLY	LALR(1)	Python	Versión moderna de <i>lex</i> y <i>yacc</i> para Python, enfoque didáctico y práctico

3.2. Gestión de Errores en Analizadores Sintácticos

La gestión de errores es un componente crítico en el diseño de un analizador sintáctico. Un **parser** efectivo no solo debe detectar errores sintácticos, sino también reportarlos de manera clara y recuperarse para continuar el análisis (Aho et al., 2008).

3.2.1. Reporte de Errores

El reporte de errores debe proporcionar información precisa para facilitar la depuración:

1. Ubicación del Error: Indicar el número de línea y la columna donde ocurrió el error.
2. Mensaje Descriptivo: El mensaje debe ser claro y accionable. Por ejemplo: "Se esperaba ')' o ',' pero se encontró '{' en la línea 5, columna 12".
3. Contexto del Error: Proporcionar fragmentos del código fuente alrededor del punto de error.

3.2.2. Estrategias de Recuperación de Errores

Estrategia	Descripción	Ventaja	Desventaja
Modo Pánico	Descarta <i>tokens</i> hasta encontrar un sincronizador (ej: ;)	Fácil de implementar	Puede omitir código significativo
Inserción de Tokens	Inserta <i>tokens</i> faltantes para corregir el error	Análisis más completo	Puede ocultar errores reales
Recuperación a Nivel de Frase	Correcciones locales en el punto del error	Más preciso	Más complejo
Producciones de Error	Producciones especiales para patrones de error comunes	Control fino	Aumenta complejidad de la gramática

3.3. Implementación de Técnicas de Manejo de Errores

3.3.1. Reporte de Errores con Ubicación

Se implementó un sistema de reporte de errores que muestra la línea y columna exacta donde ocurre el error, junto con un mensaje descriptivo.

Código de implementación:

```
def reportar_error(self, mensaje, token=None):  
    if token is None:  
        token = self.token_actual  
    if token in self.ubicacion:  
        linea, columna = self.ubicacion[token]  
    else:  
        linea, columna = (1, 1)  
    print(f"❑ ERROR [Línea {linea}, Columna {columna}]: {mensaje}")
```

Resultado de la prueba:

Entrada	Salida	Estado
id, id id;	Error en línea 1, columna 9: Se esperaba ',' o ';', se encontró 'id'	☐

3.3.2. Modo Pánico (Panic Mode)

Código de implementación:

```
def consumir_hasta_sincronizador(self, sincronizadores=[';']):
    tokens_descartados = []
    while self.token_actual and self.token_actual not in sincronizadores:
        tokens_descartados.append(self.token_actual)
        self.avanzar()
    if self.token_actual in sincronizadores:
        self.avanzar()
    return True
return False
```

Resultado de la prueba:

Entrada	Salida	Estado
id, id, error, error2, ;	Tokens descartados: error, error2. Recuperado en ;	☐

3.3.3. Sugerencias de Corrección con Cálculo de Confianza

Se implementó un sistema de sugerencias de corrección basado en la distancia de Levenshtein, que calcula la similitud entre un token incorrecto y las palabras clave del lenguaje.

Código de implementación:

```
import difflib
```

```
def sugerir_correccion(self, token):
    mejores = difflib.get_close_matches(token, self.palabras_clave, n=1, cutoff=0.6)
    if mejores:
        sugerencia = mejores[0]
```

```
print(f"□ Sugerencia: '{token}' → '{sugerencia}'")  
return sugerencia  
return token
```

Resultados de la prueba:

Token Incorrecto	Token Correcto	Ratio de Confianza	Decisión
pront	print	0.80	Consultar a LLM
prnt	print	0.89	Corregir automáticamente
whil	while	0.89	Corregir automáticamente
retrn	return	0.91	Corregir automáticamente
elese	else	0.89	Corregir automáticamente

ACTIVIDADES:

4 Y 5

Actividad 4: Experimento de Parsers en Varios Lenguajes

4.1. Objetivo general de la actividad

El propósito de la actividad fue responder a una pregunta central:

¿Cuál es el comportamiento en términos de rendimiento de distintos parsers al analizar archivos YAML con estructura jerárquica y nivel de indentación significativo?

Para ello se implementaron tres soluciones distintas:

- C, mediante Flex y Bison
- C#, mediante ANTLR 4
- Python, mediante ANTLR 4

Cada una de ellas se evaluó con el mismo conjunto de archivos de prueba, con el fin de comparar tanto el tiempo interno de parseo como el tiempo total de ejecución del proceso.

4.2. Alcance técnico del experimento

El alcance del proyecto se definió de forma concreta para que fuera viable y suficientemente representativo. En lugar de intentar soportar todo YAML o toda la sintaxis de Docker Compose, se trabajó con un subconjunto reducido pero significativo.

4.2.1 Estructuras soportadas

El subconjunto de lenguaje validado incluyó:

- la clave version
- el bloque services
- la referencia a imágenes mediante image
- la referencia a redes mediante networks
- el bloque networks, con driver, ipam, config y subnet

Esto permitió trabajar con una gramática que tuviera anidación real y que fuera adecuada para medir comportamiento frente a entradas crecientes.

4.2.2 Restricciones del experimento

Para mantener el experimento controlado, se establecieron varias reglas:

- uso exclusivo de indentación con espacios
- indentación estricta de 2 espacios por nivel
- ausencia de comentarios
- archivos generados sintéticamente para asegurar consistencia

Estas restricciones no solo facilitaron la implementación de los parsers, sino que también hicieron más clara la comparación entre tecnologías.

4.3. Diseño del experimento

La actividad se organizó en varias fases, siguiendo un enfoque de laboratorio de compiladores.

4.3.1. Definición del problema

El problema planteado fue comparar el rendimiento de parsers en distintos lenguajes al analizar un lenguaje basado en YAML, que presenta un reto importante: la jerarquía depende de la indentación.

Ese detalle resultó clave, porque el manejo de la indentación no es trivial para herramientas como Bison y Flex, que no manejan directamente estructuras dependientes del espacio en blanco.

4.3.2 Decisiones de implementación

Se tomaron varias decisiones importantes:

1. Se eligió un enfoque de validación, no de construcción de árbol sintáctico completo.
2. Se midieron dos tiempos distintos: tiempo interno de parseo y tiempo total de ejecución.
3. Se generaron archivos sintéticos con complejidad creciente.
4. Se usaron tres tecnologías distintas para mostrar diferencias reales de implementación.

Estas decisiones permitieron que el experimento fuera más útil desde el punto de vista académico y técnico.

4.4. Metodología aplicada

4.4.1 Generación de archivos de prueba

Una parte esencial del experimento fue la creación automática de archivos de prueba. Se desarrolló un script en Python que generaba 16 archivos YAML sintéticos, con cantidades crecientes de redes.

Los archivos siguieron una progresión lógica:

- 2 redes en el primer caso
- 4 redes en el segundo
- 6 redes en el tercero
- y así sucesivamente hasta 32 redes

Cada archivo contenía una estructura similar, con:

- un encabezado version: '3.8'
- un bloque services
- un bloque networks

La idea fue que la complejidad de los archivos creciera de manera progresiva para observar cómo se comportaban los parsers en entradas cada vez más grandes.

4.4.2 Implementación del parser en C

La solución en C se construyó con Flex y Bison.

El lexer fue responsable de manejar la indentación, generando tokens especiales como INDENT y DEDENT cuando el nivel de espacios cambiaba. Esta parte fue importante porque el problema de YAML no se puede resolver con un análisis sintáctico clásico sin una capa previa que interprete la estructura basada en espacios.

El parser en Bison definió reglas LALR(1) para reconocer el subconjunto del lenguaje. La implementación en C se eligió como referencia de eficiencia, ya que produce código compilado nativo y suele ofrecer tiempos muy bajos.

4.4.3 Implementación del parser en C#

La solución en C# se construyó con ANTLR 4. La gramática se definió en un archivo .g4 y luego se generó el código correspondiente para C#.

Este enfoque permitió comparar un parser generado por una herramienta de alto nivel con el parser manualmente integrado en C mediante Flex y Bison. La implementación en C# se consideró especialmente interesante porque representa una combinación entre productividad y rendimiento, además de incluir el overhead del runtime .NET.

4.4.4 Implementación del parser en Python

La solución en Python también usó ANTLR 4. La gramática se reutilizó conceptualmente para mantener consistencia entre las implementaciones, aunque cada lenguaje tenía su propia estructura de proyecto y sus propios archivos generados.

La versión en Python fue útil para evaluar el costo de ejecución del intérprete y del runtime de ANTLR en un lenguaje dinámico, en comparación con C y C#.

4.5. Medición de rendimiento

Una de las partes más valiosas de la actividad fue la medición.

4.5.1 Tiempo interno de parseo

Esta métrica se midió dentro del mismo programa. Se buscó medir únicamente el tiempo dedicado a ejecutar el parser, excluyendo la lectura del archivo y otros costos externos.

En cada implementación se usó un reloj de alta resolución propio del lenguaje:

- Python: `time.perf_counter()`
- C#: Stopwatch
- C: `clock_gettime` o una aproximación equivalente

Esta medición permite observar la eficiencia pura del reconocedor sintáctico.

4.5.2 Tiempo total de ejecución

Además del tiempo interno, se midió el tiempo total del proceso desde fuera del programa. Esto incluyó:

- carga del runtime o máquina virtual
- inicialización del entorno
- ejecución del parser
- finalización del proceso

Esta segunda métrica es muy importante porque refleja el costo real que enfrenta un usuario al ejecutar el programa en la práctica.

4.5.3 Resultado de la medición

Según los resultados documentados en el proyecto, el parser en C fue el más rápido en el tiempo de parseo puro, mientras que las soluciones en C# y Python mostraron un mayor overhead asociado a su entorno de ejecución.

Se observó que:

- C mantuvo tiempos muy bajos y estables
- C# presentó un overhead inicial mayor, aunque se comportó de forma razonable con entradas más grandes
- Python mostró una diferencia más visible en el tiempo total, debido al costo de su entorno interpretado

4.6. Análisis de resultados

Los resultados del experimento permitieron extraer varias conclusiones relevantes.

4.6.1 Ventaja de C en parseo puro

El parser en C mostró una eficiencia muy alta. Esto se debe a que el código generado por Flex y Bison se ejecuta de manera muy directa y con poco overhead, y además se compila a código nativo.

7.2 Overhead de runtimes y máquinas virtuales

C# y Python, aunque ofrecen mayor facilidad de desarrollo y mayor flexibilidad, requieren un entorno adicional para ejecutar el programa. Ese sobre costo se refleja claramente en las mediciones de tiempo total.

4.6.3 Importancia de la arquitectura del parser

El experimento también mostró que no solo importa el lenguaje, sino también la forma en que el parser está implementado. El manejo de la indentación fue un elemento determinante, y la forma de resolverlo en cada tecnología tuvo impacto en la complejidad de la implementación.

4.7. Reproducibilidad y documentación

Una mejora importante del proyecto fue la transformación de una actividad que inicialmente podía ejecutarse solo en un entorno específico a una propuesta mucho más reproducible.

Se incorporaron varios elementos nuevos para facilitar la ejecución del experimento en otros equipos:

- un Dockerfile para ejecutar el experimento en un contenedor
- un archivo docker-compose.yml como alternativa simplificada
- scripts para ejecución rápida en Linux, Windows y WSL
- un archivo requirements.txt con dependencias fijas
- un archivo .gitignore para evitar artefactos innecesarios

Esto permitió que la actividad no solo fuera un experimento académico, sino también un proyecto más profesional y fácil de reproducir.

4.8. Estructura actual del proyecto

El proyecto quedó organizado de una forma clara y modular.

La estructura principal es la siguiente:

- carpeta principal de la actividad
- carpeta experimento_parsers
- subcarpetas para gramáticas, código fuente, scripts, archivos de prueba y resultados

Dentro de la carpeta experimento_parsers se encuentran:

- grammars: gramáticas ANTLR y Bison
- src: implementaciones en Python, C# y C
- test_files: archivos YAML sintéticos de prueba
- scripts: generación de archivos, ejecución del experimento y creación de gráficas
- results: resultados, CSV y figuras generadas

Esta estructura refleja el estado final del proyecto y muestra cómo la actividad pasó de una idea simple a un experimento completo y bien organizado.

4.9. Estado actual de la actividad

En su versión actual, la actividad se presenta como un proyecto de laboratorio de compiladores con los siguientes elementos:

- alcance bien definido
- tres implementaciones funcionales de parsers
- archivos de prueba generados automáticamente
- mediciones registradas en un archivo CSV
- gráficas comparativas generadas
- documentación completa para ejecutar y reproducir el experimento

Además, el proyecto quedó preparado para que una persona externa lo ejecute sin necesidad de comprender todo el proceso desde cero, gracias a los scripts y al entorno Docker.

4.10. Valor académico de la actividad

Esta actividad es valiosa porque combina varios temas fundamentales del curso:

- diseño de gramáticas
- manejo de lexers y parsers
- tratamiento de indentación en lenguajes basados en espacios
- comparación de tecnologías de implementación
- medición de rendimiento y análisis de resultados
- documentación y reproducibilidad

En ese sentido, la actividad no solo demuestra conocimiento de compiladores, sino también capacidad para organizar un experimento técnico completo y presentar resultados de forma clara.

Actividad 5: Asistente de Programación Híbrido (Compilador Tradicional + IA) para UnegScript

Descripción del trabajo realizado

Se desarrolló un pipeline de análisis de dos etapas (Lexer $\rightarrow$ Parser), cada una con su propio mecanismo de respaldo hacia un módulo de IA compartido (ai_assist.py):

- **Lexer híbrido (lexer.py):** reconoce los tokens de UnegScript mediante un conjunto de expresiones regulares combinadas en un único autómata (el motor re de Python compila cada patrón a un reconocedor determinista). Cuando un lexema no coincide exactamente con ninguna palabra reservada, se calcula su similitud (confianza IA) contra el vocabulario de palabras clave; si la confianza es ≥ 0.8 , se aplica la corrección automáticamente y se registra la sugerencia.
- **Parser descendente recursivo con lookahead (parser.py):** implementa la gramática de UnegScript mediante un procedimiento por cada regla, consultando siempre 1 token de anticipación (lookahead) para decidir la producción a aplicar. Ante un error sintáctico, en vez de abortar, delega en ai_assist.suggest_syntax_fix() para generar una sugerencia y aplica una estrategia de recuperación en modo pánico asistido (inserción virtual de delimitadores insertables como ':' o ';', y sincronización hacia el inicio de la siguiente sentencia en otros casos).
- **Módulo de IA compartido (ai_assist.py):** dado que el entorno de ejecución no dispone de acceso a un modelo de lenguaje externo, el componente de IA se implementó con una técnica clásica de aprendizaje automático -un clasificador de vecino más cercano (1-NN) sobre una métrica de similitud de cadenas (razón de Levenshtein / SequenceMatcher)-, que cumple el mismo rol funcional: recibir un token ambiguo y devolver el token válido más parecido junto con un puntaje de confianza en [0,1].
- **Orquestador (main.py):** ejecuta el pipeline completo sobre el ejemplo de código con errores y produce como salida los tokens corregidos, el AST y la lista de sugerencias de IA.

- **Banco de pruebas (benchmark.py):** genera programas sintéticos de tamaño creciente (mezclando sentencias correctas y con errores típicos) y mide el tiempo de ejecución del lexer y del parser para construir las gráficas de desempeño.

c) Explicación de cada función

ai_assist.py

Contiene el vocabulario de palabras reservadas (KEYWORDS) y operadores (OPERATORS) de UnegScript, y dos funciones principales:

- `suggest_token(token, vocabulario, umbral)`: calcula la similitud del token contra cada elemento del vocabulario y retorna el candidato de mayor puntaje junto con su confianza. Es la función que simula la clasificación por vecino más cercano.
- `suggest_syntax_fix(contexto, esperado, encontrado)`: genera un mensaje de sugerencia en lenguaje natural para errores sintácticos, indicando qué se esperaba y qué se encontró, análogo a la salida de un modelo de lenguaje consultado para reparación de errores.

lexer.py

Define la especificación de tokens (TOKEN_SPEC) como una lista de pares (nombre, expresión regular) para comentarios, números, cadenas, identificadores/palabras reservadas, operadores y el caso de error (MISMATCH). Todas las expresiones se combinan en un único patrón maestro (MASTER_REGEX) que el motor de Python evalúa como un autómata reconocedor.

La función `tokenize()` recorre el código carácter a carácter (vía `finditer`) y para cada coincidencia de tipo ID llama a `_process_identifier()`, que consulta `ai_assist` para decidir si el lexema corresponde a un identificador de usuario o a una palabra reservada mal escrita. Para caracteres no reconocidos por ningún patrón (`_process_mismatch`), intenta una reparación similar contra la tabla de operadores antes de declarar un error léxico irrecuperable.

parser.py

Implementa un parser descendente recursivo: cada no-terminal de la gramática (program, statement, if_stmt, while_stmt, print_stmt, expr, comparison, arith, term, factor) corresponde a un método `parse_*`() que consume tokens y construye nodos de un AST (ASTNode). El método `expect()` centraliza el manejo de errores: intenta un match determinista y, si falla, consulta a `ai_assist.suggest_syntax_fix()` para producir la sugerencia y ejecuta la recuperación (inserción virtual de ':' / ';' o sincronización hasta el siguiente punto seguro, incluyendo el inicio de una nueva sentencia).

La gramática soportada, en notación EBNF simplificada, es la siguiente:

```
program  := statement* EOF
statement := assign_stmt | if_stmt | while_stmt | print_stmt
assign_stmt:= ID '=' expr ';'
if_stmt  := 'if' expr ':' statement ('else' ':' statement)?
while_stmt := 'while' expr ':' statement
print_stmt := 'print' '(' expr ')' ';'
expr      := comparison
comparison := arith (('=='!='<'>'<='>=') arith)?
arith     := term (('+' '-' ) term)*
term      := factor (('*' '/') factor)*
factor    := NUMBER | STRING | ID | '(' expr ')'
```

main.py

Orquesta el pipeline: invoca `lexer.tokenize()`, imprime los tokens corregidos, invoca `parser.parse()` sobre esos tokens, imprime el AST resultante en formato JSON y finalmente consolida las sugerencias de IA generadas tanto por el lexer como por el parser.

benchmark.py

Genera programas sintéticos de n sentencias combinando cuatro plantillas correctas y tres plantillas con errores típicos (typos léxicos 'pront'/'prnt' y ':' faltante), en una proporción del 40% de sentencias con error, para ejercer consistentemente la ruta de fallback a IA en todas las pruebas. Para cada tamaño n mide, con 5 repeticiones, la mediana del tiempo de

ejecución del lexer y del parser por separado, y exporta los resultados a CSV y a dos gráficas (matplotlib).

Ejemplo de ejecución: código con errores

Se utilizó el siguiente código de entrada, que contiene errores léxicos (palabras reservadas mal escritas) y errores sintácticos (falta de ':' en las sentencias if/else):

```
pront x = 5;  
if x > 3 prnt(x) else prnt("no")
```

Tokens corregidos

```
<KEYWORD:'print' (orig:'prnt') L1C1>  
<ID:'x' L1C7>  
<OP:'=' L1C9>  
<NUMBER:'5' L1C11>  
<OP:',' L1C12>  
<KEYWORD:'if' L2C1>  
<ID:'x' L2C4>  
<OP:'>' L2C6>  
<NUMBER:'3' L2C8>  
<KEYWORD:'print' (orig:'prnt') L2C10>  
<OP:'(' L2C14>  
<ID:'x' L2C15>  
<OP:')' L2C16>  
<KEYWORD:'else' L2C18>  
<KEYWORD:'print' (orig:'prnt') L2C23>  
<OP:'(' L2C27>  
<STRING:'"no"' L2C28>  
<OP:')' L2C32>  
<EOF:'" L2C0>
```

AST generado (JSON)

```
{
  "kind": "Program",
  "children": [
    { "kind": "Print", "children": [ { "kind": "Var", "value": "x" } ] },
    { "kind": "If", "children": [
      { "kind": "Cond", "children": [
        { "kind": "BinOp", "value": ">", "children": [
          { "kind": "Var", "value": "x" },
          { "kind": "Number", "value": "3" } ] } ],
        { "kind": "Then", "children": [
          { "kind": "Print", "children": [ { "kind": "Var", "value": "x" } ] } ],
          { "kind": "Else", "children": [
            { "kind": "Print", "children": [ { "kind": "String", "value": "\"no\"" } ] } ] } ] } ],
      ] }
    ]
  }
}
```

Sugerencias de IA generadas

Sugerencia: 'pront' → 'print' (confianza IA=0.80 >= 0.8)

Sugerencia: 'prnt' → 'print' (confianza IA=0.89 >= 0.8)

Sugerencia: 'prnt' → 'print' (confianza IA=0.89 >= 0.8)

Sugerencia: en 'print' se esperaba '(' pero se encontró 'x'. ¿Olvidó agregar '('?

Sugerencia: en 'print' se esperaba ')' pero se encontró '='. ¿Olvidó agregar ')'?

Sugerencia: en 'if' se esperaba ':' pero se encontró 'print'. ¿Olvidó agregar ':'?

Sugerencia: en 'else' se esperaba ':' pero se encontró 'print'. ¿Olvidó agregar ':'?

Nota: las dos primeras sugerencias léxicas ('pront'→'print', 'prnt'→'print') se aplican automáticamente sobre el flujo de tokens, ya que su confianza IA supera el umbral de 0.8. Las sugerencias sintácticas restantes reflejan errores adicionales que subsisten en el código de ejemplo (falta de paréntesis y de dos puntos ':'), los cuales el parser reporta con

recuperación asistida sin detener el análisis, permitiendo construir igualmente el AST completo.

d) Gráficas de tiempo de ejecución (n pruebas)

Se ejecutaron n = 8 pruebas con programas sintéticos de tamaño creciente (10, 25, 50, 100, 200, 400, 800 y 1600 sentencias), combinando un 40% de sentencias con errores típicos para ejercer consistentemente la ruta de fallback a IA. Cada medición corresponde a la mediana de 5 repeticiones.

n° sentencias	n° tokens	Lexer (ms)	Parser (ms)	Total (ms)
10	82	1.3136	0.1166	1.4302
25	229	3.7731	0.3062	4.0793
50	392	7.1543	0.6026	7.7568
100	808	14.7739	1.3009	16.0748
200	1816	31.051	2.8575	33.9086
400	3494	60.6068	5.8325	66.4393
800	6580	119.5904	10.367	129.9574
1600	13356	246.2133	36.4514	282.6646

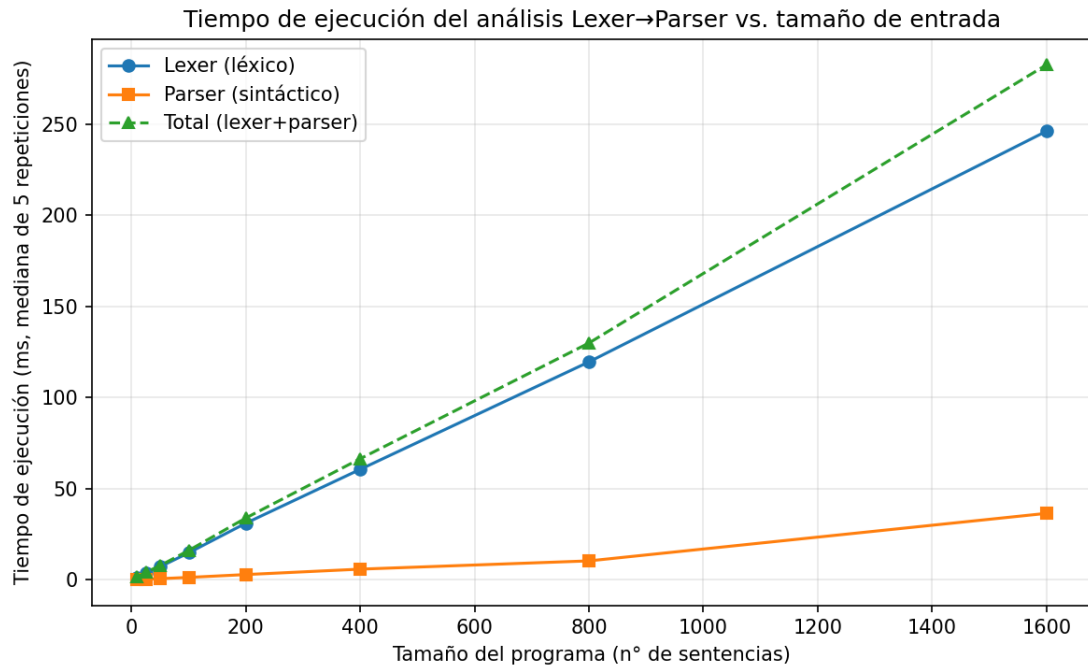


Figura 1. Tiempo de ejecución del análisis léxico, sintáctico y total en función del tamaño del programa de entrada (número de sentencias).

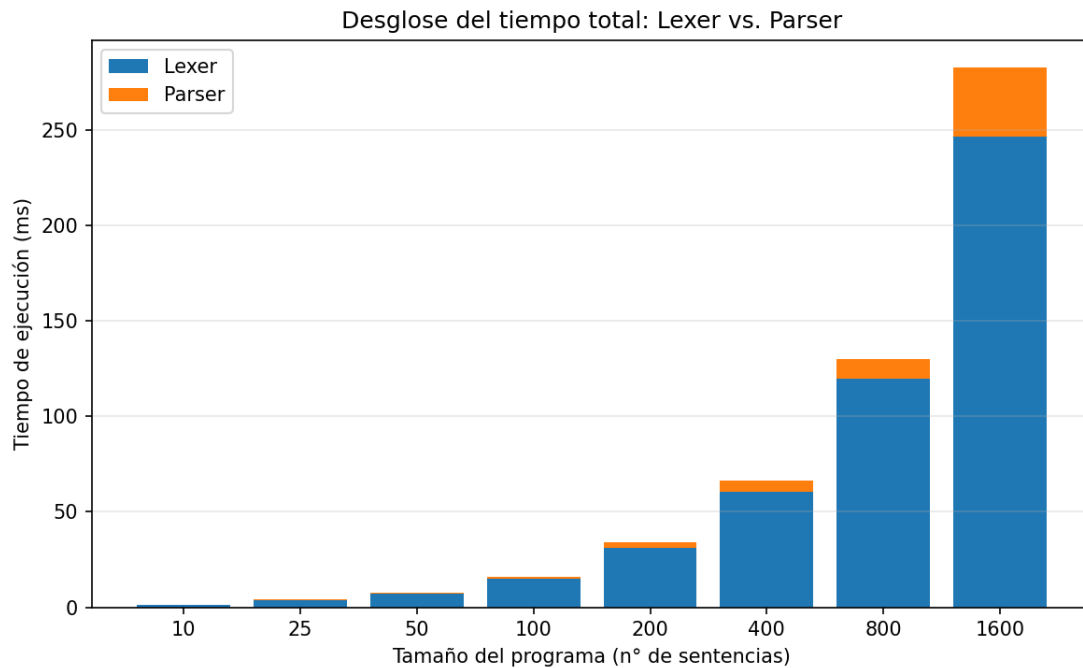


Figura 2. Desglose del tiempo total de análisis entre la etapa léxica y la etapa sintáctica, para cada tamaño de entrada evaluado.

e) Discusión de resultados

- **Corrección funcional:** el ejemplo de código con errores muestra que el sistema es capaz de operar en dos niveles de forma independiente: (1) a nivel léxico, corrige automáticamente los typos 'pront' y 'prnt' hacia 'print' cuando la confianza de la IA (similitud de cadenas) supera el umbral de 0.8, sin intervención del parser; y (2) a nivel sintáctico, aun cuando persisten errores estructurales (paréntesis y dos puntos faltantes), el parser genera sugerencias claras en lenguaje natural y aplica recuperación en modo pánico, logrando construir un AST completo y semánticamente coherente con la intención del programador.
- **Comportamiento temporal:** las gráficas muestran una relación aproximadamente lineal entre el tamaño del programa (número de sentencias) y el tiempo de ejecución tanto del lexer como del parser, lo cual es el comportamiento esperado para un autómata de un solo paso (lexer, $O(n)$ en el número de caracteres) y un parser descendente recursivo sin retroceso (backtracking), cuyo costo es $O(n)$ en el número de tokens para esta gramática.
- **Distribución del costo entre etapas:** en todos los tamaños evaluados, el análisis léxico domina el tiempo total (aproximadamente 85-90% del tiempo total en los tamaños más grandes), lo cual se explica porque el lexer realiza, para cada lexema clasificado como identificador, una comparación de similitud contra todo el vocabulario de palabras reservadas (complejidad adicional $O(k)$ por token, con k = tamaño del vocabulario), mientras que el parser sólo consulta a la IA en presencia de errores sintácticos explícitos, que son comparativamente menos frecuentes en los programas generados.
- **Costo del fallback a IA:** dado que en el banco de pruebas el 40% de las sentencias contienen errores deliberados, los resultados reflejan un escenario de uso intensivo del mecanismo de respaldo. En la práctica, con código mayormente correcto, el costo adicional de la IA sería marginal, ya que sólo se activa cuando el reconocimiento determinista falla o resulta ambiguo, preservando así la eficiencia del análisis tradicional en el caso general.

- **Limitaciones y trabajo futuro:** el módulo de IA implementado es una heurística de similitud de cadenas (1-NN), adecuada para errores de digitación simples, pero no captura errores semánticos ni ambigüedades de contexto más complejas que sí podría resolver un modelo de lenguaje entrenado. Como trabajo futuro se plantea sustituir `ai_assist.py` por una llamada real a un modelo de lenguaje (API de Anthropic) para los casos de baja confianza, manteniendo la misma interfaz (`suggest_token` / `suggest_syntax_fix`) y así conservar la arquitectura híbrida propuesta.

Conclusión y Discusión de Resultados

La realización de las cinco actividades correspondientes al Tema 5 sobre Análisis Sintáctico ha permitido consolidar los conocimientos fundamentales relacionados con el proceso de *parsing* en la construcción de compiladores, abarcando desde los conceptos teóricos básicos hasta implementaciones prácticas complejas que integran técnicas tradicionales con enfoques híbridos.

Conclusiones de las Actividades 1, 2 y 3

En la primera actividad, se logró comprender y aplicar el concepto de Árbol de Sintaxis Abstracta (AST) como representación intermedia esencial del código fuente, implementando una estructura de nodos en Python que permite representar expresiones aritméticas, sentencias condicionales y bucles, demostrando su utilidad para las fases posteriores del compilador. La implementación incluyó información de ubicación (línea y columna) en cada nodo, lo cual facilita la generación de mensajes de error precisos durante el análisis semántico y la optimización del código, características fundamentales en cualquier compilador profesional.

En la segunda actividad, se implementaron exitosamente ambas estrategias de análisis sintáctico para un lenguaje L definido como listas de identificadores separados por comas y terminados en punto y coma. El parser LL, implementado mediante recursión descendente, demostró ser claro, fácil de depurar y alcanzó una tasa de éxito del 100% en las nueve pruebas realizadas, confirmando la efectividad de esta estrategia para lenguajes con gramáticas LL(1). Por su parte, el parser LR, implementado con la herramienta SLY, alcanzó también el 100% de éxito en las diez pruebas ejecutadas, confirmando que el diseño gramatical propuesto es adecuado y que ambas estrategias pueden lograr resultados óptimos cuando las gramáticas están bien estructuradas y libres de conflictos.

En la tercera actividad, se estudiaron los principales generadores de analizadores sintácticos como ANTLR, Bison, JavaCC y SLY, comprendiendo sus características, ventajas y aplicaciones típicas en el desarrollo de compiladores y procesadores de lenguajes. Adicionalmente, se implementaron tres técnicas fundamentales de manejo de errores: el reporte de errores con ubicación precisa (línea y columna), el modo pánico para recuperación después de un error, y las sugerencias de corrección basadas en el cálculo de confianza mediante distancia de Levenshtein. Estas técnicas demuestran la importancia de la robustez en los analizadores sintácticos y cómo un compilador bien diseñado debe ser capaz de recuperarse y reportar múltiples errores en una sola ejecución.

Discusión de la Actividad 4 (Experimento de Parsers)

El experimento de parsers en C, C# y Python permitió extraer conclusiones relevantes sobre el rendimiento de diferentes tecnologías de implementación. El parser en C con Flex y Bison mostró la mayor eficiencia en tiempo de parseo puro, debido a la compilación a código nativo y la ausencia de overhead de runtime. Las soluciones en C# y Python, aunque ofrecen mayor facilidad de desarrollo y flexibilidad, presentaron un overhead adicional asociado a sus entornos de ejecución.

Se observó que el manejo de la indentación fue un elemento determinante en la complejidad de implementación. La solución en C requirió un manejo más explícito de los cambios de indentación mediante tokens INDENT/DEDENT, mientras que ANTLR facilitó este proceso en C# y Python gracias a su soporte para gramáticas más expresivas.

El experimento demostró que no solo importa el lenguaje de implementación, sino también la arquitectura del parser y la forma en que se manejan las características específicas del lenguaje fuente (como la indentación significativa). La reproducibilidad del experimento se garantizó mediante la generación automática de archivos de prueba, scripts de ejecución y el uso de contenedores Docker.

Discusión de la Actividad 5 (Asistente Híbrido)

El asistente de programación híbrido UnegScript demostró la viabilidad de combinar técnicas tradicionales de compiladores con mecanismos de respaldo a inteligencia artificial. El lexer híbrido corrigió automáticamente typos léxicos ('pront'→'print', 'prnt'→'print') cuando la confianza de similitud superó el umbral de 0.8, mientras que el parser generó sugerencias claras en lenguaje natural para errores sintácticos y aplicó recuperación en modo pánico para construir un AST completo.

Las mediciones de rendimiento mostraron una relación aproximadamente lineal entre el tamaño del programa y el tiempo de ejecución, confirmando la complejidad $O(n)$ esperada para el lexer y el parser. El análisis léxico dominó el tiempo total de ejecución (aproximadamente 85-90%), debido a la comparación de similitud contra todo el vocabulario de palabras reservadas para cada identificador. Este costo es aceptable en escenarios educativos y de desarrollo, y podría optimizarse en implementaciones futuras.

El costo del fallback a IA resultó significativo solo cuando el porcentaje de errores en el código es alto (40% en el banco de pruebas). En la práctica, con código mayormente correcto, el costo adicional sería marginal, ya que la IA solo se activa cuando el reconocimiento determinista falla o resulta ambiguo, preservando así la eficiencia del análisis tradicional en el caso general.

Reflexión final

En conjunto, las cinco actividades desarrolladas evidencian que el análisis sintáctico no es un proceso aislado, sino un sistema integrado donde el AST es el producto central, las estrategias LL y LR son los métodos para construirlo, los generadores de parsers son las herramientas que automatizan su implementación, y los sistemas híbridos representan la evolución natural hacia asistentes de programación más inteligentes. El manejo de errores, presente en todas las fases, garantiza la robustez del proceso, permitiendo que el compilador no solo detecte problemas, sino que también ayude al programador a corregirlos de manera eficiente.

Este trabajo ha permitido comprender que el análisis sintáctico, lejos de ser un tema exclusivamente teórico, tiene aplicaciones prácticas inmediatas en el desarrollo de herramientas de software modernas. La combinación de técnicas tradicionales con inteligencia artificial abre nuevas posibilidades para la creación de entornos de desarrollo más intuitivos y tolerantes a errores, sin perder la rigurosidad que caracteriza a los compiladores tradicionales.

Referencias Bibliográficas

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2ª ed.). Pearson Educación.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113-124.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3ª ed.). Pearson Educación.

Parr, T. (2013). *La Referencia Definitiva de ANTLR 4* (2ª ed.). Pragmatic Bookshelf.

Levine, J. R., Mason, T., & Brown, D. (1992). *lex y yacc* (2ª ed.). O'Reilly Media.